Test – xUnit

xUnit är ett populärt ramverk för enhetstestning i .NET-applikationer.

1. Skapa testprojekt

Lägg till ett nytt xUnit-testprojekt i din befintliga lösning. Högerklicka på lösningen i Solution Explorer, välj **Add > New Project**, sök efter "xUnit Test Project" och skapa det. Ge det ett namn som "**FundamentalsPluralSight.Tests**".

2. Installera NuGet-paket

Se till att ditt testprojekt har nödvändiga paket. Högerklicka på testprojektet, välj **Manage NuGet Packages** och installera:

- **Xunit**
  - Kärnpaketet med assertions och testattribut som [Fact] och [Theory]

  För .NET 8+ / xUnit v3

  Om du använder nyare versioner:

  xunit.v3.core istället för xunit

  xunit.v3.assert för assertions
- **xunit.runner.visualstudio**
  – Adapter för att köra tester i Visual Studios Test Explorer
- **Microsoft.NET.Test.Sdk**
  - Ger testkörningsinfrastruktur och integration med dotnet test
- **coverlet.collector** (för code coverage, valfritt)

Alternativt, använd Package Manager Console: Install-Package xunit osv.
dotnet add package xunit
dotnet add package Microsoft.NET.Test.Sdk
dotnet add package xunit.runner.visualstudio

3. Lägg till referens

Högerklicka på testprojektet i Solution Explorer, välj Add > Project Reference och markera din console app-projekt. Detta låter testen komma åt din kod.

Skriv din första test

Ersätt innehållet i **UnitTest1.cs** med ett exempel. Antag att din console app har en **klass Calculator** med metoden **Add(int a, int b)**:

```
using Xunit;
using using BookTestingDemo;   // Ersätt med ditt projekt-namn

public class CalculatorTests
{
  [Fact]
  public void Add_TwoNumbers_ReturnsSum()
  {
     // Arrange
     var calc = new Calculator();
     int a = 2;
     int b = 3;

     // Act
     int result = calc.Add(a, b);

     // Assert
     Assert.Equal(5, result);

  }
}
```

Använd **[Fact]** för enkla tester och **Assert-metoder** för validering.

4. **Kör testen**

Öppna **Test Explorer** (Test > Test Explorer). Klicka **Run All** för att köra alla tester. Gröna ikoner betyder pass, röda betyder fail – klicka på dem för detaljer.

5. **Nästa steg**

Lär dig **[Theory]** och **[InlineData]** för **parametriserade** tester,
eller **ITestOutputHelper** för att skriva ut **debug**-info.

Kör från kommandoraden med **dotnet test** i projektmappen.

Ja, .NET 10 (som släpptes november 2025) fungerar utmärkt med xUnit v3, som är den nyare varianten av xUnit. Den klassiska xUnit v2 (senaste 2.9.3) är fortfarande aktuell, men v3 är framtiden med bättre prestanda och stöd för Microsoft Testing Platform (MTP).

Vad är Microsoft Testing Platform (MTP)?

Använd dessa NuGet-paket i ditt testprojekt:

```
dotnet add package xunit.v3.core
dotnet add package xunit.v3.assert
dotnet add package Microsoft.NET.Test.Sdk
dotnet add package xunit.runner.visualstudio  // För Visual Studio
```

Viktigt för .NET 10

.NET 10 SDK stöder primärt MTP (Microsoft Testing Platform) i stället för traditionell VSTest. Lägg till denna global.json i lösningsroten:

```
{
  "sdk": {
    "version": "10.0.100",
    "test": { "runner": "Microsoft.Testing.Platform" }
  }
}
```

Detta säkerställer att dotnet test och Visual Studio använder MTP v2.

Rekommendation

Byt till **xUnit v3** för din .NET 10 console app:

1. Avinstallera gamla xunit-paket
2. Installera xunit.v3.core + xunit.v3.assert
3. Lägg till global.json med MTP
4. Testa med dotnet test och Visual Studio Test Explorer

v3 har bättre prestanda och är optimerad för moderna .NET-versioner som din .NET 10-miljö!

Migrationstips

1. Byt paket: xunit → xunit.v3.core + xunit.v3.assert
2. Ändra OutputType till Exe i testprojektet
3. Fixa async void → async Task
4. Testa med dotnet test (MTP) eller behåll VSTest-adapter

v3 är **framtidssäker** för .NET 10 men kräver koduppdateringar – v2 fungerar fortfarande utmärkt för stabila projekt.

Annat exempel:

```
[Fact]
0 references
public void TestToString3()
{
    // Arrange
    Person person = new Person("Stina", "Svensson", 30, 500);
    person.IncreaseSalary(10);
    // Act
    string result = person.ToString();
    // Assert
    Assert.Equal("Stina Svensson receives 550,00 dollars.", result);
}
```

MOCKNING:

För mock-objekt använder man vanligtvis komplement som

Moq, NSubstitute eller FakeItEasy.

xUnit = test runner och testramverk.

Mockning = separat bibliotek för att ersätta beroenden i tester.

Vanligaste valet i .NET är ofta Moq, men NSubstitute är också populärt.

Genomgång MS Test

- Som ni tidigare har sett finns det en mängd olika s.k. Testing Tools…
  (Junit, Nunit, Jmockit, EMMA, PHPUnit m.fl.)

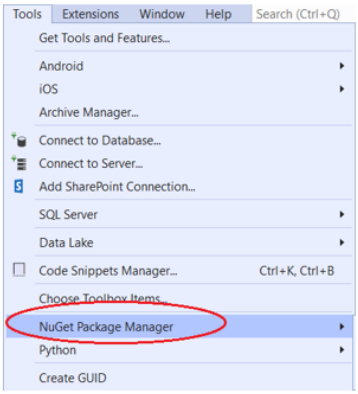
- MS test är alltså ett ramverk som gör det möjligt att testa kod utan inblandning av tredjeparts verktyg.

- Det hjälper till att skriva effektiva enhetstester för att testa sina applikationer.

Många versioner av MS test tillgängliga

- NuGet Gallery | MSTest.TestFramework 2.2.4
- Första versionen: 1.0.0 släpptes 16-05-27
- MS Test.Framework 2.2.4 är den senaste versionen som släpptes för 15 timmar sedan ☺ (21-05-25)
- I Visual Studio 2019 ör MSTest installerad här:
  C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\Common7\IDE\MStest

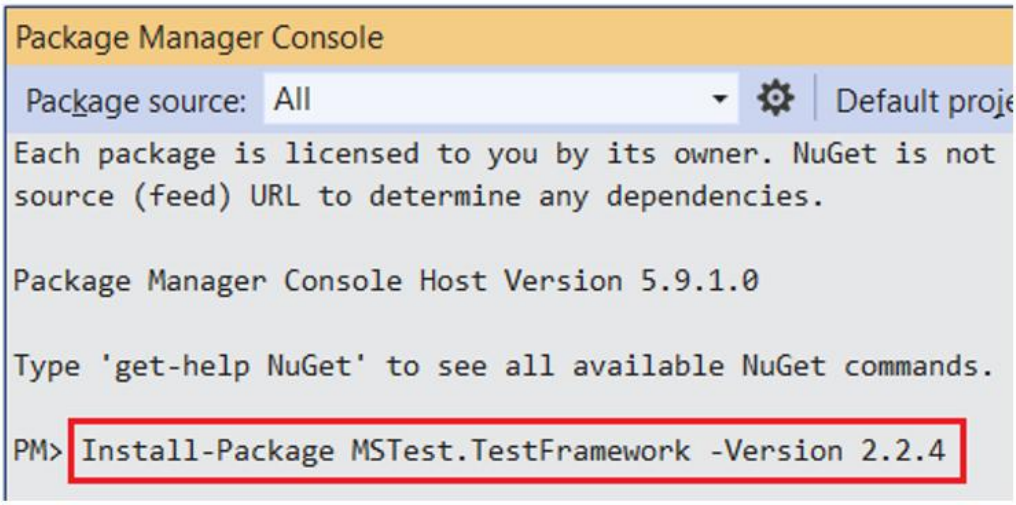
MSTest Version 2 krävs i .NET Core

- MSTest version 2 installeras via Nuget Package Manager i Visual Studio
- 

- Which to write tests targeting .NET Framework, .NET Core and ASP.NET Core on Windows, Linux, and Mac.

Installera MSTest Version 2 på två olika sätt:

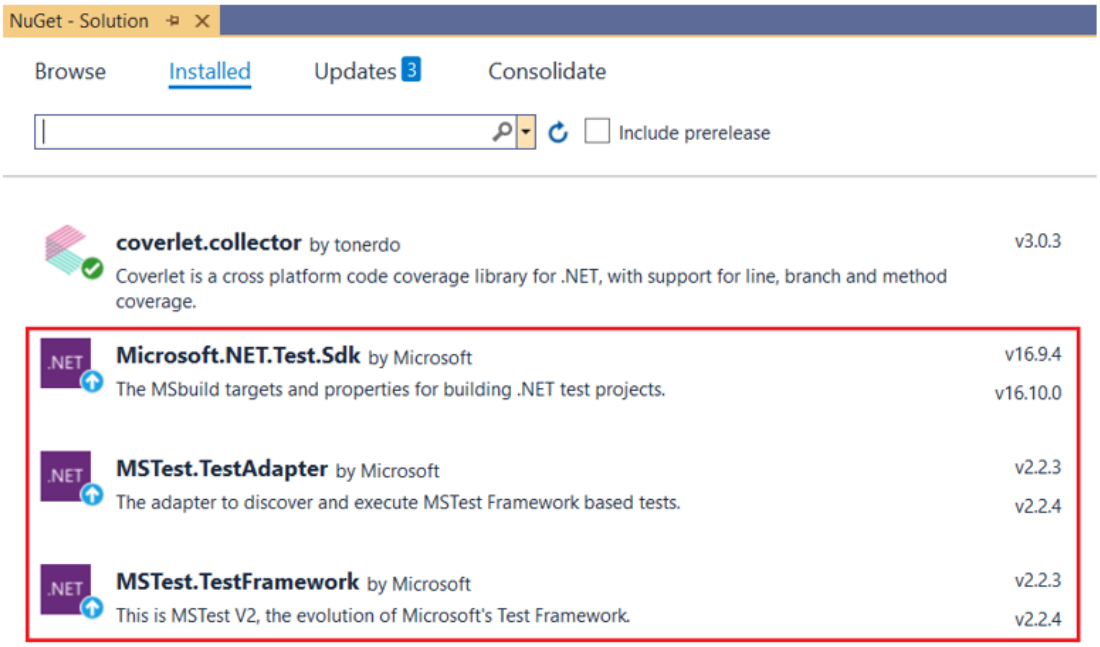
1. Via Nuget Package Manager =>
   Package Manager Console

- 

Installera MSTest Version 2 på två olika sätt:

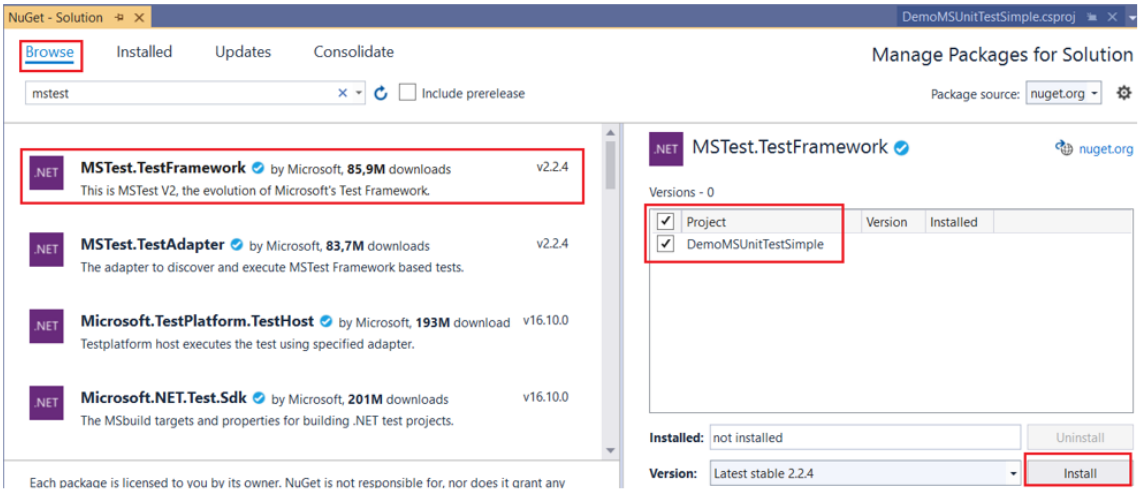
2. Via Nuget Package Manager => Manage Nuget Packages for Solution

Nedanstående paket ska installeras.



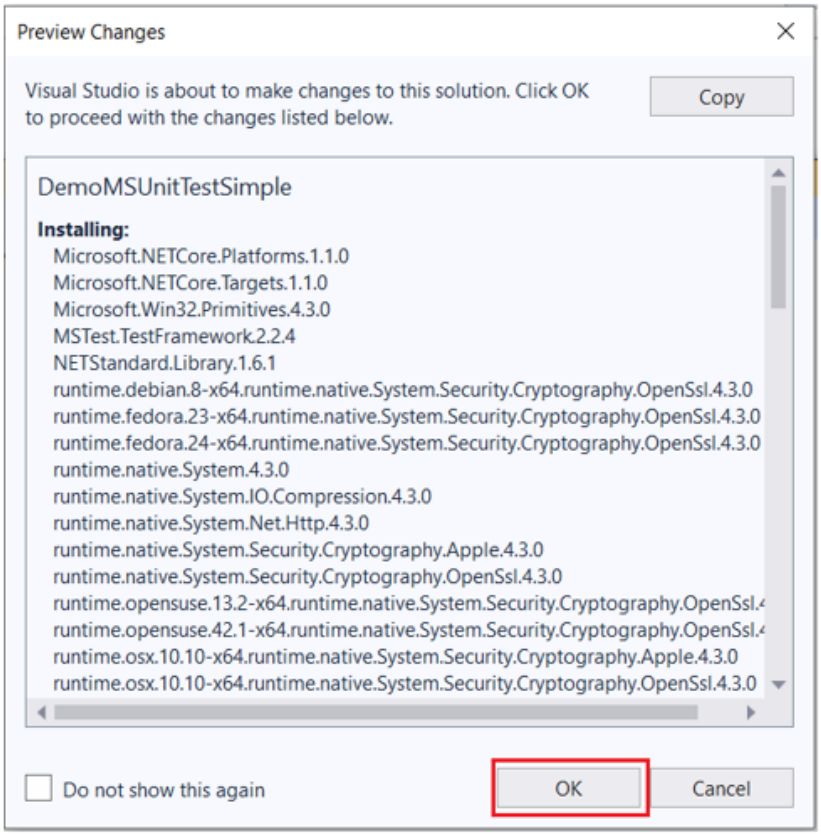
Installation av MSTest Version 2: (1/3)

Via Nuget Package Manager =>

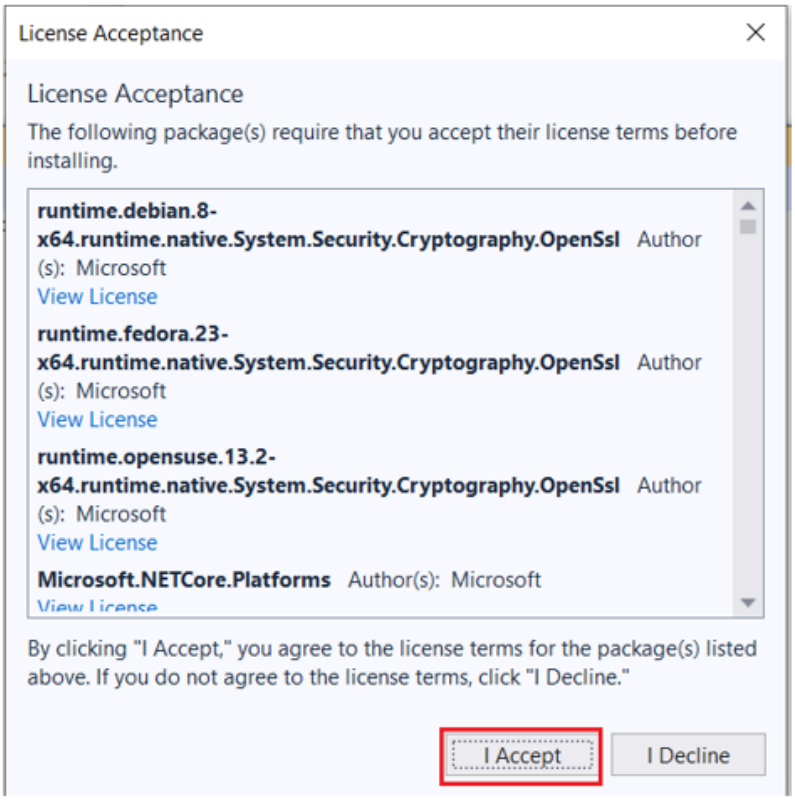


Installation av MSTest Version 2: (2/3)

Via Nuget Package Manager =>



Installation av MSTest Version 2: (3/3)
Via Nuget Package Manager =>



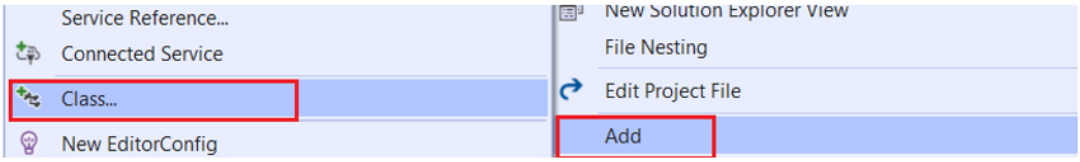
Installation av MSTest Version 2: (3/3)
Via Nuget Package Manager =>

Gör samma installation för paketen:

Mstest.TestAdapter

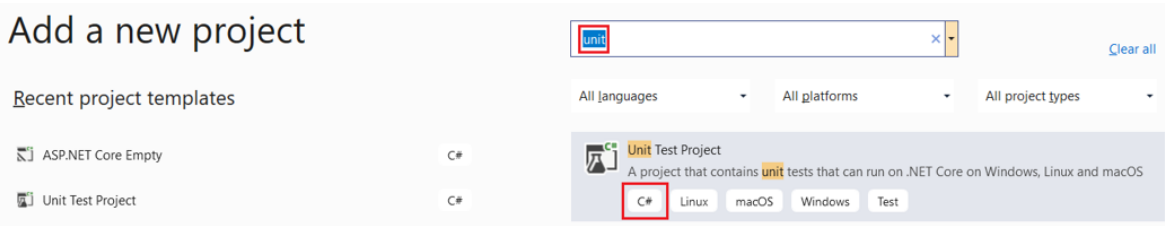
Hur testar man en applikation med MSTest?

- Öppna en befintlig applikation eller skapa en ny (Asp.Net Core Empty)
- Anta att man ska testa metoder i en klass, tex klassen SimpleMaths som innehåller några metoder:
  Add(double num1, double num2)
  Substract(double num1, double num2)
  Divide(double num1, double num2)
  Multiply(double num1, double num2)
- Skapa en ny klass genom att hö.kl. på projektnamnet i Solution Explorer =>



För att kunna testa dessa metoder, skapar man ett nytt Unit Test Projekt:

Genom att högerklicka på Solution => Add => New Project
Väljer Unit Test Project C#
Namn på detta nya project ska vara samma som klassnamnet.Test
I detta fallet: SimpleMaths.Test

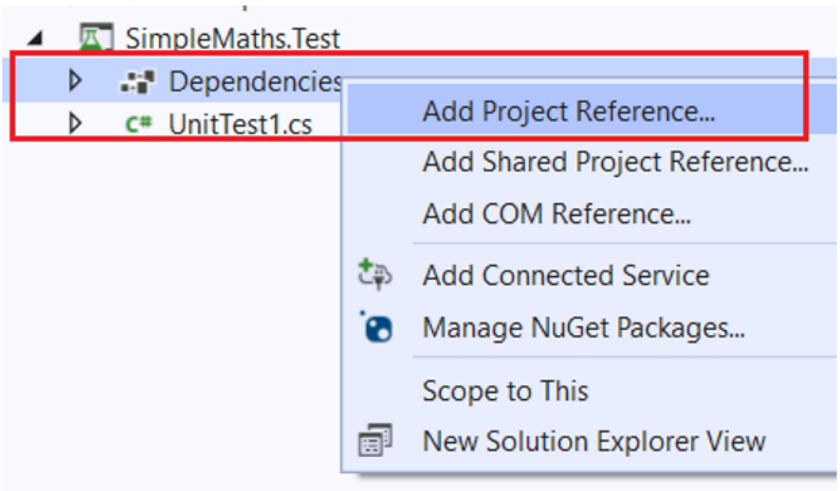


Här ser ni kodblocket som Unit Test Project skapar:

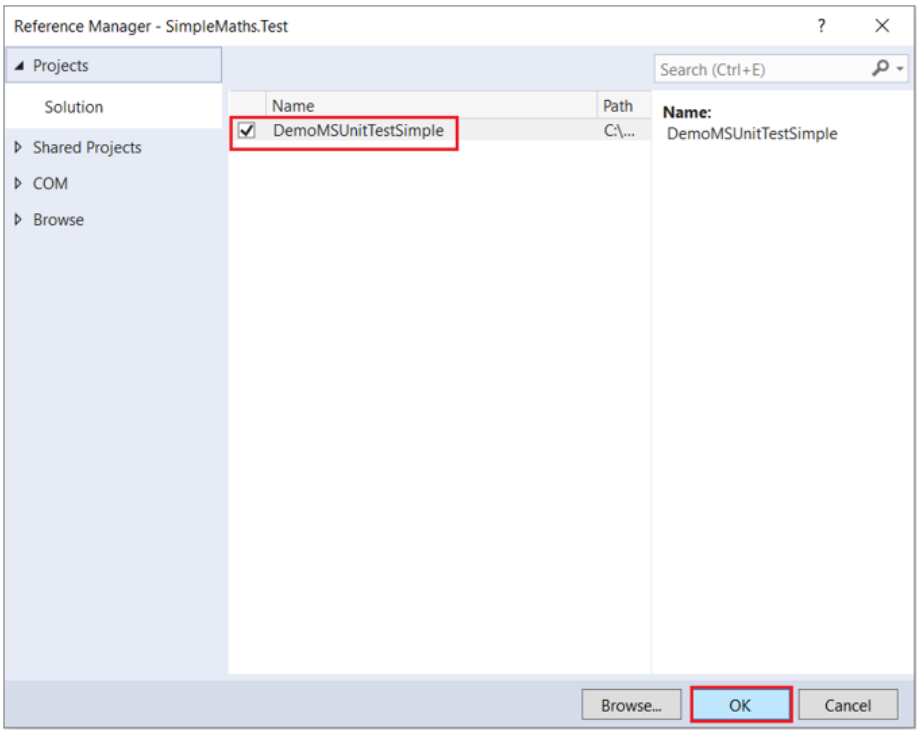
```
using Microsoft.VisualStudio.TestTools.UnitTesting;
namespace SimpleMaths.Test
{
    TestClass]
    public class UnitTest1
    {
        [TestMethod]

        public void TestMethod1()
        {
        }
    }
}
```

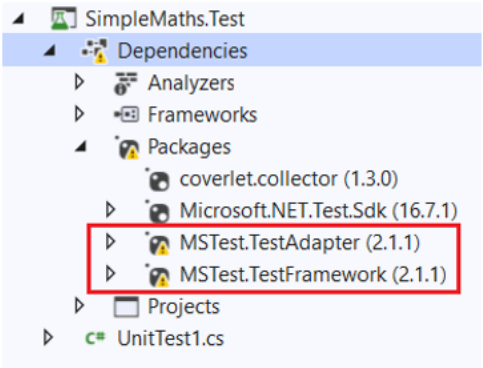
Skapa en referens mellan detta Unittest projekt och metoderna i klassen SimpleMaths

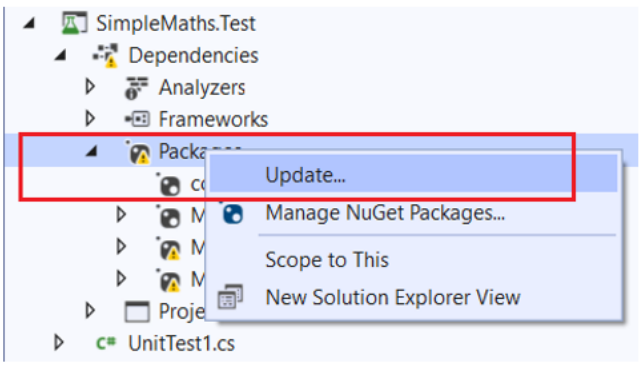


Skapa en referens mellan detta Unittest projekt och metoderna i klassen SimpleMaths



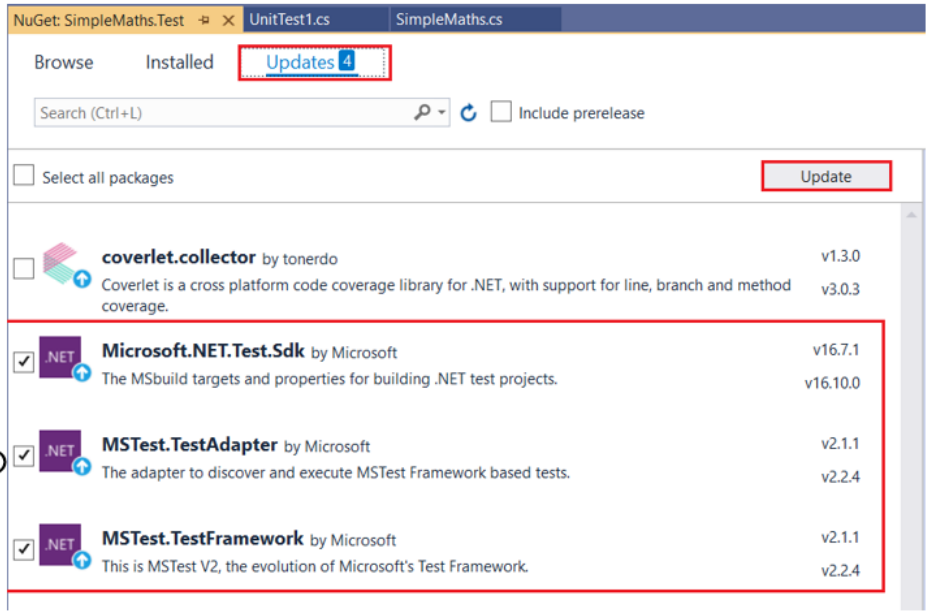
Uppdatera eventuellt paketen under Dependencies





Uppdatera eventuellt paketen under Dependencies

Välj o



Skriv testmetoder för respektive metod i klassen SimpleMaths

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
namespace SimpleMaths.Test
{
    [TestClass]
    public class UnitTest1
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

MSTest använder olika typer av attribut, T.ex. [TestClass], [TestMethod]

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
namespace SimpleMaths.Test
{
    [TestClass]
    public class UnitTest1
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

Skriv testmethoden under attributet [TestMethod]
i filen UnitTest1.cs

```
Namespace DemoMSUnittestSimple {        // Obs samma namespace som
  [TestClass]                           // SimpleMath.cs
   public class UnitTest1
   {
      [TestMethod]
      public void Test_AddMethod()
      {
         SimpleMaths sm = new SimpleMaths();  // Arrange initializes objects
         double res = sm.Add(10, 40);   // Act invokes the method under test
         Assert.AreEqual(res, 50);     // The Assert section verifies that the action of
                                          the method
      }                                // under test behaves as expected
   }
}
```

Skriv testmethoden under attributet [TestMethod] i filen UnitTest1.cs

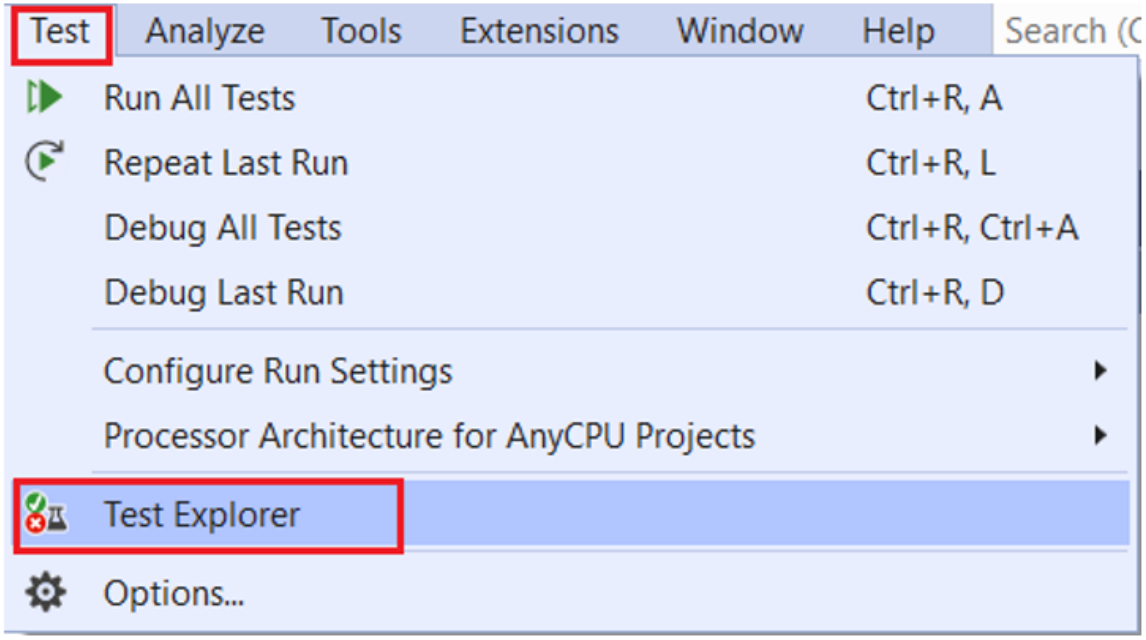
```
[TestClass]
   public class UnitTest1
   {
      [TestMethod]
      public void Test_AddMethod()
      {
         SimpleMaths sm = new SimpleMaths();  // Arrange initializes objects
         double res = sm.Add(10, 40);   // Act invokes the method under test
         Assert.AreEqual(res, 50);     // The Assert section verifies that the action of
                                          the method
                                       // under test behaves as expected
      }
   }
```

The AAA in MSTest

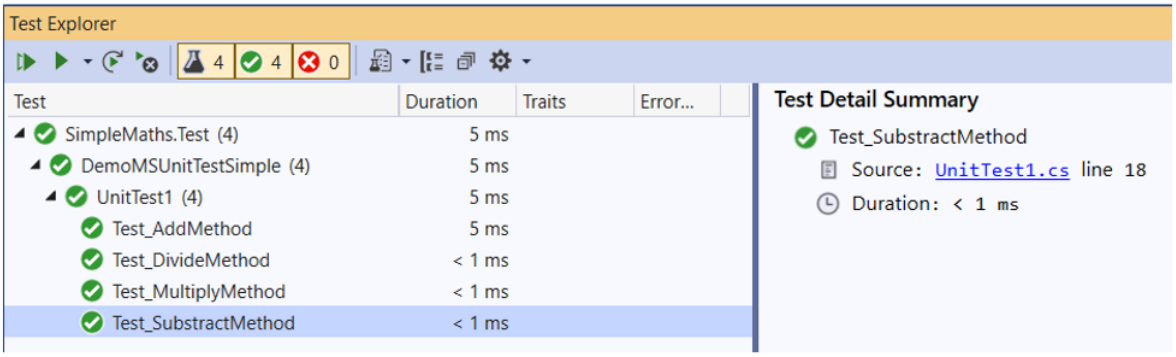
- The AAA (Arrange, Act, Assert) pattern is a common way of writing unit tests for a method under test.
- The **Arrange** section of a unit test method initializes objects and sets the value of the data that is passed to the method under test.
- The **Act** section invokes the method under test with the arranged parameters.
- The **Assert** section verifies that the action of the method under test behaves as expected.

Run tests in Test Explorer

För att köra sina tester, välj menyn



Test Explorer



Utforska Assert mer ingående

[MSTest v2: Exploring asserts - Meziantou's blog]

Utforska MSTest Attributes or Annotations mer ingående

[Most Complete MSTest Framework Tutorial Using .Net Core (lambdatest.com)]

ANNOTATION	DESCRIPTION
[TestInitialize]	Marks a method that should be called before each test method. One such method should be present per test class.
[TestCleanup]	Marks a method that should be called after each test method. One such method should be present per test class.
[TestClass]	Marks a class that contains tests.
[TestMethod]	Marks a method, i.e., an actual test case in the test class.
[DataRow]	Allows setting the values of the parameters of the test. More than one [DataRow] annotation can be present in the code.